SotilityBridgeAdapter.sol

SotilityBridgeAdapter.sol

v1: Multi-chain token bridge registry + forwarder

Modular. Secure. Cross-network ready.

Features:

Register remote chains and destination contract addresses

Send token bridge requests

Emit clean cross-chain events

Add emergency flagging for future AI extensions

Notes:

This version emits events for LayerZero / Axelar / CCIP relayer hooks

You could plug this into a backend script or Chainlink Automation to process bridges

Easily extensible with AI validation & rate-limit rules

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title SotilityBridgeAdapter
 * @dev Bridges SST/SUG/SOT tokens across multiple chains. Future-compatible with AI + fraud detection.
 */
contract SotilityBridgeAdapter is AccessControl {
    bytes32 public constant BRIDGE_OPERATOR_ROLE = keccak256("BRIDGE_OPERATOR_ROLE");

    struct Destination {
        string chainName;
        address destinationBridge;
        uint64 chainId;
        bool enabled;
    }

    IERC20 public token;
    mapping(uint64 => Destination) public destinations;

    event DestinationAdded(uint64 chainId, string chainName, address bridge);
    event TokenBridged(address indexed sender, uint64 destinationChainId, uint256 amount, string reason);
    event BridgeSuspicious(address indexed sender, string reason);

    constructor(address _token) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        token = IERC20(_token);
    }

    function addDestination(
        uint64 chainId,
        string calldata name,
        address bridgeAddress
    ) external onlyRole(DEFAULT_ADMIN_ROLE) {
        destinations[chainId] = Destination({
            chainName: name,
            destinationBridge: bridgeAddress,
            chainId: chainId,
            enabled: true
        });

        emit DestinationAdded(chainId, name, bridgeAddress);
    }

    function bridgeToken(uint64 chainId, uint256 amount, string calldata reason)
        external
    {
        Destination memory dest = destinations[chainId];
        require(dest.enabled, "Destination disabled");

        require(token.transferFrom(msg.sender, address(this), amount), "Transfer failed");

        // Placeholder logic: event emits for LayerZero or relayer bot to pick up
        emit TokenBridged(msg.sender, chainId, amount, reason);
    }

    // Extension hook: flag suspicious activity
    function flagSuspicious(address sender, string calldata reason) external onlyRole(BRIDGE_OPERATOR_ROLE) {
        emit BridgeSuspicious(sender, reason);
    }

    function getDestination(uint64 chainId) external view returns (Destination memory) {
        return destinations[chainId];
    }
}
```

Notein
Scan the QR code to download Notein